HostelOS

Smart Student Hostel Management Platform

Complete Advanced Project Roadmap

Commercial multi-tenant SaaS • Web + Installable PWA • FastAPI backend • AI/ML analytics

PRODUCT GOAL
Build a production-oriented platform to manage multiple hostel organizations, branches, rooms, students, staff, fees, attendance, services and analytics from one secure system.

DELIVERY APPROACH
Start with a modular monolith and API-first architecture. Prove core workflows, security and tenant isolation before considering service extraction.

ROADMAP ESTIMATE
Approximately 16–20 weeks for a focused team of 3–5 people; longer for a solo developer or a fully audited commercial launch.

Prepared as an implementation blueprint — requirements, architecture, data model, delivery phases, testing, deployment and launch gates.

Roadmap at a glance

Use this document as the master delivery plan. Each phase ends with a reviewable artifact and a clear exit condition.

01 Product vision, scope and release strategy

02 Personas, roles and core user journeys

03 Advanced feature map and module boundaries

04 Recommended architecture and technology stack

05 Data architecture and multi-tenant database plan

06 API design and backend implementation

07 Frontend, PWA and user experience

08 AI/ML roadmap and responsible use

09 Security, privacy and operational controls

10 Testing and quality engineering

11 DevOps, deployment and observability

12 16–20 week implementation schedule

13 Acceptance criteria, risks and launch checklist

14 Suggested repository structure and first sprint

Important: this is a product roadmap, not a claim that the system is already implemented. Validate local hostel policies, payment provider requirements, privacy obligations and institutional workflows before production launch.

01. Product vision, scope and release strategy

Product name: HostelOS — Smart Student Hostel Management System.

Business model: SaaS subscription for private hostels, university hostels and hostel operators managing one or more branches. Each customer organization is a tenant; each tenant can own multiple hostels/branches.

Core promise: One reliable operational system from admission and bed allocation through billing, attendance, complaints, checkout and management reporting.

Release strategy

- **Foundation release:** secure tenant onboarding, user accounts, branch setup, room/bed inventory, student records and admissions.
- **Operations release:** allocations, fee invoices, payment recording, attendance/gate logs, leave requests, complaints and notifications.
- **Intelligence release:** management dashboards, forecasts, anomaly flags and AI-assisted triage after sufficient clean operational data exists.
- **Commercial release:** subscription plans, tenant usage limits, support workflows, backups, monitoring, audit evidence and documented onboarding.

Keep the first commercial version focused. Do not delay the core product for advanced AI, native mobile apps, microservices or every possible integration.

02. Personas, roles and core user journeys

Role	Main responsibilities	Typical permissions
Super Admin (platform)	SaaS operations, tenant support, plans, global health	Manage tenants/plans; support access must be audited and time-limited
Hostel Owner / Admin	Organization settings, branches, staff, policies, reports	Full access within own organization
Warden	Daily hostel operations, student welfare, gate and leave workflows	Operational access scoped to assigned hostel
Accountant	Invoices, payments, refunds, dues and finance reports	Finance-only actions; no unrestricted student document access
Security Guard	Entry/exit, visitor logs, QR verification	Minimal gate workflow; no finance or sensitive profile access
Mess Manager	Menus, meal plans, stock and meal counts	Mess and inventory scope
Maintenance Staff	Assigned work orders and updates	Only assigned tickets and required location details
Student	Profile, invoices, payments, leave, complaints, notices	Own records only
Guardian	Approved student updates and permitted payments	Only linked student and explicitly shared information

Key journeys

- **Tenant onboarding:** owner signs up → verifies email → creates organization and first hostel → configures buildings/rooms/beds → invites staff → selects subscription.
- **Admission:** application submitted → documents reviewed → decision recorded → student profile created → bed allocated transactionally → invoice generated → student invited.
- **Monthly billing:** fee schedule generates invoices → student receives notice → payment is recorded or gateway webhook is verified → receipt and ledger updated.

- **Outpass:** student requests leave → warden approves/rejects → gate staff verifies approved pass → actual exit/return events are recorded.
- **Complaint:** student submits category, description and optional photo → ticket assigned → status and SLA tracked → resolution confirmed → feedback captured.

03. Advanced feature map and module boundaries

Module	Advanced scope
Platform & SaaS	Tenant onboarding, branch management, plans, subscription status, usage quotas, tenant settings, support tools.
Identity & access	Email verification, password reset, optional MFA, session/device management, RBAC, scoped permissions.
Admissions & students	Applications, review checklist, student lifecycle, guardian links, documents, emergency contacts, status history.
Accommodation	Buildings, floors, room types, rooms, beds, availability, allocation history, transfers, checkout and maintenance blocks.
Finance	Fee structures, recurring charges, invoices, line items, discounts, penalties, payments, refunds, receipts, reconciliation and ageing.
Attendance & gate	QR/NFC/manual check-in, gate logs, visitor registration, outpass, late-return alerts and offline-safe capture.
Student services	Complaints, maintenance tickets, announcements, notices, lost-and-found, student feedback.
Mess & inventory	Meal plans, menu calendar, meal attendance, stock items, purchase records, stock movement and low-stock alerts.
Staff & HR	Staff profiles, assignments, shifts, attendance, leave and optional payroll integration.
Communication	In-app notifications, email/SMS/WhatsApp provider adapters, templates, delivery status and user preferences.
Analytics	Occupancy, revenue, dues, collection rate, admissions funnel, complaints SLA, attendance trends and branch comparisons.
AI intelligence	Occupancy forecast, fee-risk prioritization, complaint classification, maintenance forecasting and anomaly review.

Build modules as independent domain packages inside one backend deployment. Each module owns its rules and services; modules communicate through explicit service interfaces and domain events rather than importing each other's database internals.

04. Recommended architecture and technology stack

Recommended shape: responsive Next.js application + API-first FastAPI modular monolith + PostgreSQL + Redis/Celery + separately deployable AI inference/training components. Web and installable PWA use the same versioned API.

Layer	Recommended technology	Implementation notes
Frontend	Next.js, React, TypeScript	App Router; server/client components deliberately; feature-based UI; accessible forms and tables.
UI system	Tailwind CSS, shadcn/ui	Design tokens, reusable layouts, role-aware navigation, responsive dashboards.
API/backend	Python, FastAPI, Pydantic	Versioned REST API; OpenAPI contract; dependency-injected services; consistent errors.
ORM & migrations	SQLAlchemy 2.x, Alembic	Explicit constraints, reviewed migrations, transaction boundaries and rollback plan.

Layer	Recommended technology	Implementation notes
Database	PostgreSQL	Tenant-scoped composite keys, indexes, constraints, RLS defense in depth.
Cache & jobs	Redis, Celery	Email/SMS, invoice generation, scheduled reminders, report exports and ML batch jobs.
Realtime	FastAPI WebSockets or managed pub/sub	Only for live dashboards, gate status and operational notifications where needed.
Files	S3-compatible object storage	Private buckets, short-lived signed URLs, MIME/size validation, malware scanning where feasible.
AI/ML	scikit-learn, XGBoost, pandas	Start with interpretable baselines; separate training from inference; model registry and monitoring.
Quality	Pytest, Playwright, Ruff, mypy	Unit, integration, API contract, end-to-end and static checks.
Delivery	Docker, Nginx, GitHub Actions	Environment parity, automated tests, migration gate, health checks and rollback.

Architecture decision: do not begin with microservices. A modular monolith is faster to develop, test and operate. Extract a service only when scaling, team ownership or reliability data proves a real need.

05. Data architecture and multi-tenant database plan

Tenant model

- organizations is the tenant root; hostels/branches belong to an organization. Every tenant-owned business row carries organization_id, and where appropriate hostel_id.
- Derive tenant identity from the authenticated principal and validated membership. Never trust organization_id supplied by a browser request as proof of access.
- Use organization-scoped composite foreign keys (for example organization_id + parent_id) so records cannot accidentally reference another tenant's rows.
- Apply PostgreSQL Row-Level Security as defense in depth. Set tenant context per transaction/connection safely; test pooled-connection reset behavior.
- Add audit_logs for sensitive actions, including actor, tenant, action, target, timestamp, request correlation ID and carefully redacted metadata.

Core entity groups

Domain	Core tables / entities
Tenant & identity	organizations, hostels, users, memberships, roles, permissions, role_permissions, refresh_tokens, invitations
Admissions	students, guardians, student_guardians, applications, application_documents, student_documents
Buildings & beds	buildings, floors, room_types, rooms, beds, allocations, allocation_events, room_blocks
Finance	fee_structures, fee_items, invoices, invoice_items, payments, refunds, receipts, payment_webhook_events
Operations	attendance_records, gate_logs, leave_requests, visitors, visitor_logs, complaints, complaint_assignments, maintenance_records
Mess & staff	meal_plans, menus, meal_attendance, inventory_items, stock_movements, employees, shifts, staff_attendance
Platform & AI	notifications, announcements, audit_logs, prediction_records, model_versions, subscriptions, subscription_plans

Critical relational design rules

- **Bed allocation:** allocations reference beds, not rooms. Keep allocation history; do not overwrite history when a student moves.
- **Prevent double booking:** use database-level constraints where practical (for PostgreSQL, an exclusion constraint over bed_id and a daterange can prevent overlapping active allocations). Define whether checkout dates are inclusive or exclusive.
- **Correct floor ownership:** floors should be scoped to a hostel and reference a building in that same hostel; rooms should reference a floor with matching hostel scope.
- **Capacity:** treat beds as the authoritative capacity source, or enforce room capacity against bed count with a transactional service/trigger and reconciliation report.
- **Money:** use integer minor units or fixed-precision Decimal; store currency code; never use binary float for balances. Record gateway references idempotently.
- **Deletion:** prefer archival/status transitions for financial and audit entities. Define retention and anonymization policies.

The ERD and migrations must be reviewed before production. Do not copy an early draft schema without validating composite foreign keys, tenant scope, date-range semantics and transaction safety.

06. API design and backend implementation

API conventions

- Prefix endpoints with /api/v1; publish OpenAPI; use consistent pagination, filtering, sorting and error envelopes.
- Use request/response schemas distinct from ORM models. Validate input at the boundary and enforce business rules in services.
- Use idempotency keys for payment creation, webhook handling, invoice generation and other retryable financial operations.
- Use database transactions for admission + allocation + invoice creation; fail atomically if any step fails.
- Use optimistic concurrency/version fields for high-contention edits; return clear conflict responses.
- Keep authorization checks at the service/use-case boundary, not only in frontend navigation.

Suggested endpoint families

Area	Example routes
Auth	POST /auth/login, POST /auth/refresh, POST /auth/logout, POST /auth/password/forgot
Tenants	GET/POST /organizations, GET/PATCH /organizations/{id}, POST /hostels
Students	GET/POST /students, GET/PATCH /students/{id}, POST /students/{id}/documents
Admissions	POST /applications, POST /applications/{id}/review, POST /applications/{id}/approve
Rooms	GET /rooms/availability, POST /allocations, POST /allocations/{id}/transfer, POST /allocations/{id}/checkout
Finance	POST /invoices, GET /invoices, POST /payments, POST /webhooks/{provider}, GET /finance/ledger
Operations	POST /leave-requests, POST /gate-events, POST /complaints, PATCH /complaints/{id}/status
Analytics	GET /analytics/occupancy, GET /analytics/revenue, GET /analytics/arrears, GET /analytics/forecast

07. Frontend, PWA and user experience

Build the interface around workflows, not just CRUD screens.

- **Role-based workspaces:** separate dashboards and navigation for owner/admin, warden, accountant, guard, mess manager, staff, student and guardian.

- **Admin experience:** occupancy cards, overdue invoices, admissions queue, complaints awaiting action, recent gate events and branch filters.
- **Student portal:** profile, room/bed details, dues and receipts, leave/outpass, complaints, notices and support contacts.
- **Guardian portal:** only explicitly permitted student information, payment links and approved notifications.
- **Forms:** autosave where safe, document upload progress, clear validation, confirmation for destructive actions and accessible keyboard navigation.
- **Tables:** server-side pagination/filtering, saved views for staff, export permissions and empty/loading/error states.
- **PWA:** install prompt, app manifest, icons, service worker, offline shell and carefully scoped cached content. Do not cache sensitive financial/student data by default.
- **Responsive:** prioritize mobile gate and student workflows; support desktop-heavy finance and administration screens.

Suggested frontend route groups

app/(public) • login, signup, pricing, help
app/(platform) • super-admin console
app/(tenant) • organization and branch admin
app/(operations) • warden, guard, accountant, mess and maintenance
app/(portal) • student and guardian
features/ • auth, students, admissions, rooms, finance, attendance, complaints, mess, analytics

08. AI/ML roadmap and responsible use

AI should be introduced only after workflows produce reliable, permissioned and sufficiently representative data. Start with decision support, show confidence and evidence, and keep a human accountable for consequential actions.

Use case	Data / method	Output and safeguards
Occupancy forecasting	Historical check-ins, allocations, checkouts, seasonality; baseline moving average then tree/time-series model	Expected occupancy by hostel/room type; show forecast interval and data coverage.
Fee-risk prioritization	Invoice ageing, payment history, due dates, permitted engagement signals; calibrated classification	Prioritize human follow-up; explain top contributing factors; never auto-deny accommodation.
Complaint classification	Ticket text/category; TF-IDF baseline then approved NLP model	Suggest category/priority/queue; staff confirms; monitor language and category bias.
Maintenance prediction	Asset/service history, issue frequency, downtime	Flag assets for inspection; do not claim certainty from sparse data.
Attendance anomaly review	Gate events and expected schedules; rules first, anomaly detection later	Surface unusual patterns for review; avoid automatic disciplinary action.
Revenue forecast	Invoice and collection time series by tenant/branch	Cash-flow planning with uncertainty bands; separate billed revenue from collected cash.

AI delivery lifecycle

- Define target and decision owner; document what the model must never decide.
- Create a data dictionary, lineage, consent/legal basis and retention rules.
- Build a rule-based or statistical baseline before complex models.
- Split data by time and tenant to avoid leakage; evaluate per hostel/segment, not only aggregate accuracy.
- Track precision/recall, calibration, drift, missingness and human override rates.
- Version datasets, features, model artifacts and evaluation reports; support rollback to a known-good model.
- Expose predictions through an internal inference interface; keep training jobs separate from live request handling.

09. Security, privacy and operational controls

Minimum commercial security baseline

- TLS everywhere; secure, HttpOnly, SameSite cookies where appropriate; short-lived access tokens and rotating refresh tokens with revocation.
- Password hashing with Argon2id or a maintained password-hashing library; optional MFA for privileged roles.
- RBAC plus resource scope: role alone is insufficient; verify organization, hostel, assigned work and record ownership.
- Rate limiting and abuse protection on authentication, password reset, uploads and public endpoints.
- Validate file type by content, size limits, private object storage, signed download URLs and malware scanning where feasible.
- Verify webhook signatures, timestamp/replay protection and idempotency; never trust a browser redirect as proof of payment.
- Encrypt backups; test restore; restrict production access; rotate secrets; keep secrets out of Git and logs.
- Audit high-risk actions: permission changes, student data exports, refunds, allocation overrides, impersonation and support access.
- Minimize sensitive data; define retention, deletion, export and incident response procedures.
- Use dependency scanning, container image scanning, static analysis and periodic penetration testing.

Privacy-by-design note

Student and guardian data can be sensitive. Collect only what is necessary, make guardian visibility configurable and legally reviewed, avoid exposing identity documents in broad staff views, and define who can access data and for what operational reason.

10. Testing and quality engineering

Test level	What to verify	Exit evidence
Unit	Pricing, invoice totals, permission decisions, allocation date rules, status transitions	Fast deterministic suite; high coverage of critical domain rules.
Integration	PostgreSQL constraints, tenant scoping, Redis jobs, object storage adapter, payment webhook flow	Real service dependencies in CI containers; rollback and retry tests.
API contract	Schemas, auth failures, pagination, filtering, idempotency, version compatibility	OpenAPI validation and endpoint contract tests.
End-to-end	Tenant signup → branch setup → admission → allocation → invoice → payment → receipt	Playwright flows across roles and mobile viewport.
Security	Cross-tenant IDOR, privilege escalation, CSRF/XSS, rate limits, upload abuse, token replay	Security test report and resolved critical/high findings.
Performance	Dashboard queries, student search, invoice generation, concurrent bed allocation	Load profile with p95 latency, error rate and DB saturation.
Recovery	Backup restore, migration rollback/forward recovery, worker restart, provider outage	Documented recovery exercise with measured RPO/RTO.

Critical negative tests

- Tenant A cannot read, update, export or infer Tenant B data by changing IDs, filters or file references.
- Two simultaneous requests cannot allocate the same bed for overlapping dates.
- Duplicate payment webhook delivery cannot create duplicate ledger entries or receipts.

- A warden cannot access finance administration unless explicitly granted.
- A guardian cannot view another student's record by guessing a student ID.
- A failed admission transaction leaves no partial allocation or invoice.

11. DevOps, deployment and observability

Environments

- **Local:** Docker Compose for PostgreSQL and Redis; seeded demo tenant; fake payment/email adapters.
- **CI:** lint, type checks, unit/integration tests, dependency scan, frontend build and migration validation on every pull request.
- **Staging:** production-like services, synthetic tenant data, payment sandbox, migration rehearsal and end-to-end smoke tests.
- **Production:** managed PostgreSQL, managed Redis or reliable service, object storage, TLS reverse proxy, application workers and scheduled backups.

Observability essentials

- Structured JSON logs with request ID, tenant ID where safe, actor ID and operation; redact tokens, passwords and sensitive student fields.
- Metrics: API latency/error rate, DB connections/slow queries, queue depth, worker failures, webhook failures, storage usage and tenant usage.
- Tracing across API → database → queue → worker for slow multi-step workflows.
- Alerts for elevated 5xx, payment reconciliation failures, queue backlog, backup failures and certificate expiry.
- Health endpoints: liveness and readiness; readiness should check dependencies required to serve traffic.

Release safety

- Use backward-compatible expand-and-contract migrations for zero/low-downtime deployments.
- Run migrations as a controlled release step, not independently in every application replica.
- Use feature flags for risky capabilities; canary or staged rollout where practical.
- Document rollback triggers, restore points and who is authorized to execute recovery.

12. 16–20 week implementation schedule

Estimate assumes a focused team: 1 product/UX owner, 1–2 frontend engineers, 1–2 backend engineers and shared QA/DevOps/ML support. A solo developer should expect a substantially longer schedule.

Timing	Phase	Main work	Exit gate
Week 1	Discovery & product definition	PRD, personas, workflow maps, MVP scope, policy questions, success metrics	Approved scope and backlog
Week 2	Architecture & UX foundation	Architecture decisions, wireframes, design system, threat model, tenant model	Reviewed architecture and clickable prototype
Weeks 3–4	Platform foundation	Repo, CI, Docker, auth, tenant onboarding, organizations, hostels, RBAC, audit baseline	Tenant isolation tests pass
Weeks 5–6	Student & accommodation	Admissions, student profiles, documents, buildings/floors/rooms/beds, allocation engine	No cross-tenant access or double booking
Weeks 7–8	Finance core	Fee structures, invoices, payment records, receipts, refunds, reconciliation groundwork	Ledger and idempotency tests pass

Timing	Phase	Main work	Exit gate
Weeks 9–10	Hostel operations	Attendance, gate logs, leave/outpass, visitors, complaints, maintenance	Role-based operational E2E flows pass
Week 11	Communication & PWA	Notifications, templates, PWA install, offline shell, responsive refinements	Mobile smoke tests pass
Week 12	Reporting & analytics	Occupancy, arrears, collection, admissions and service dashboards	Metric definitions signed off
Weeks 13–14	AI foundation	Data quality, baselines, model evaluation, prediction API, human review UI	AI is measurable and safely optional
Week 15	SaaS billing & integrations	Plans, quotas, provider adapters, webhook verification, email/SMS/payment sandbox	Subscription and webhook tests pass
Weeks 16–17	Hardening	Security tests, load tests, accessibility, backup restore, observability, documentation	No unresolved critical/high launch blockers
Weeks 18–20	Pilot & launch	Pilot tenants, staff training, feedback fixes, staged rollout, support playbook	Pilot sign-off and production readiness

13. Acceptance criteria, risks and launch checklist

MVP acceptance criteria

- A new organization can onboard, create branches and invite staff without platform-operator database intervention.
- Every tenant-owned API and file operation is tenant-scoped and covered by automated isolation tests.
- Admin can configure beds, admit a student, allocate a bed and process checkout with full history.
- Finance can create invoices, record payments, issue receipts and reconcile gateway events idempotently.
- Warden/guard can process leave and gate events with timestamped audit history.
- Students can access only their own allowed portal data; guardian access is explicitly linked and scoped.
- Dashboards use documented metric definitions and permission-aware filters.
- Backup restore, incident escalation and support processes have been rehearsed.

Major risks and mitigation

Risk	Mitigation
Scope creep	Protect MVP; maintain a change log; defer low-value modules and native apps.
Tenant data leakage	Composite FKs, RLS, service-level scope checks, cross-tenant automated tests and security review.
Incorrect financial balances	Immutable ledger events, Decimal/minor units, idempotency, reconciliation and audit trails.
Double bed allocation	Transactional allocation service plus database overlap constraint and concurrency tests.
Weak AI data	Delay model claims; use baselines, data quality checks, confidence display and human review.
Provider downtime	Retry queues, idempotent adapters, status visibility and manual fallback procedures.
Operational adoption	Pilot with real staff, role-specific training, simple workflows and feedback cycles.

Production launch checklist

- Legal review of privacy notice, terms, data processing, retention and guardian access.
- Production secrets, domain, TLS, backups, alerting and least-privilege access configured.
- Database migrations rehearsed against a production-like copy; restore drill completed.

- Payment gateway live credentials and webhook signing verified; reconciliation owner assigned.
- Admin, warden, accountant, guard and student help guides published.
- Support escalation, incident response, data export and account closure processes documented.
- Pilot feedback triaged; critical/high defects resolved; launch owner signs release checklist.

14. Suggested repository structure and first sprint

hostelos/ ■■■ frontend/ # Next.js + TypeScript PWA ■ ■■■ src/app/ # route groups and layouts ■ ■■■ src/features/ # auth, students, rooms, finance, operations ■ ■■■ src/components/ # shared UI and design system ■ ■■■ src/lib/ # API client, validation, auth helpers ■ ■■■ tests/ # Playwright and component tests ■■■ backend/ ■ ■■■ app/api/v1/ # versioned routers and endpoints ■ ■■■ app/core/ # config, security, permissions, logging ■ ■■■ app/db/ # session, metadata, tenant context ■ ■■■ app/modules/ # domain modules: students, rooms, finance... ■ ■■■ app/integrations/ # payment, email, SMS, storage adapters ■ ■■■ app/tasks/ # Celery tasks ■ ■■■ migrations/ # Alembic migrations ■ ■■■ tests/ ■■■ ai_engine/ ■ ■■■ datasets/ # metadata only; keep private data out of Git ■ ■■■ preprocessing/ ■ ■■■ training/ ■ ■■■ evaluation/ ■ ■■■ inference/ ■ ■■■ model_registry/ ■■■ infrastructure/ # Docker, Nginx, monitoring, scripts ■■■ docs/ # PRD, ADRs, ERD, API, runbooks ■■■ .github/workflows/ # CI/CD ■■■ docker-compose.yml ■■■ .env.example ■■■ README.md ■■■ LICENSE

First 10 working days — practical sprint

Day	Task	Definition of done
1	Create repository, branching rules, issue templates and docs skeleton	README, contribution rules, project board and ADR template committed.
2	Create Docker Compose for PostgreSQL and Redis; configure environment variables	One command starts local dependencies; secrets are not committed.
3	Bootstrap FastAPI, settings, structured logging and health endpoints	App starts, readiness/liveness work, config validation is tested.
4	Bootstrap Next.js, design tokens, app shell and responsive navigation	Authenticated shell and public landing/login screens render.
5	Create SQLAlchemy base, Alembic and organization/hostel models	Initial migration applies and rolls back on clean database.
6	Implement user, membership and role foundations	Tenant membership and role assignment tests pass.
7	Implement login, refresh rotation, logout and password reset flow	Auth lifecycle tests pass; tokens are stored securely.
8	Add tenant context and organization-scoped repository/service patterns	Cross-tenant negative tests pass for initial endpoints.
9	Add audit event service and invitation workflow	Invitations expire, are single-use and actions are auditable.
10	Review threat model, architecture and sprint demo	Architecture review completed; next sprint backlog is prioritized.

Recommended build order

Security + tenant foundation → student and bed lifecycle → finance integrity → daily hostel operations → PWA and communication → analytics → AI → SaaS scale. This sequence reduces rework because advanced reporting and ML depend on trustworthy operational data.

Technology version note: choose currently supported stable releases at implementation time, pin dependencies, commit lockfiles and review security advisories. Keep .env secrets out of source control; provide only safe placeholders in .env.example.

End of roadmap